
Fitness-neutral sets are observed in systems where genotype-phenotype mapping displays a many-to-one relationship. This study highlights the phenomena of percolation where multiple genotype specimens which manifest a single phenotype originate from diverse areas of configuration space. We show that percolation is a probable feature of certain scenarios in Stephen Wolfram's minimal model for biological evolution and argue that the production of percolating fitness-neutral sets and the analysis thereof can be of use in biological studies and possibly in other relevant scientific areas.

Fitness-Neutral Sets and Percolation

What is a fitness-neutral set?

In some systems a chain of genotype specimens whose adjacent members are connected to each other by subsequent single-point mutations maps in its entirety a single phenotype. Such a chain is called a fitness-neutral set. In Stephen Wolfram's minimal model for biological evolution [1], a fitness-neutral set is a chain of cellular automata rules (the genotype) that map, for example, to a single lifetime (the phenotype). In RNA research, a fitness-neutral set is known as a neutral network (both terms will be interchangeably used in this essay), and is a chain of RNA sequences that map to a single RNA secondary structure.

What is percolation?

Do fitness-neutral sets span the entire configuration space or are they limited to small islands therein? Percolation alludes to the former observation. Schuster, Fontana, Stadler and Hofacker [2] showed that RNA sequences forming neutral networks percolate in sequence space and cited the strong implications it has on the underlying system (e.g., epistasis). Reidys, Stadler and Schuster [3] put forth a scheme of quantifying RNA neutral networks that included criteria for percolation.

Percolation: fundamental metrics and major criteria

In this work we used two neutral networks metrics:

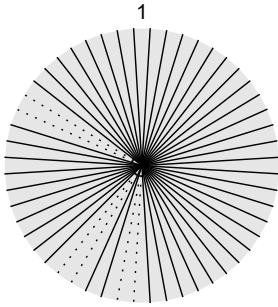
Average Local Neutrality - the mean of phenotype-conserving single-point mutations across members of a neutral network.

Network Diameter - the maximum Hamming distance between any two members of the neutral network.

And two percolation criteria imposed on them:

1. Average local neutrality of the neutral network, λ , is larger than $\lambda^* = \sqrt[\alpha-1]{\frac{1}{\alpha}}$ where α = alphabet size, e.g. $\alpha=4$ for RNA sequences and $\alpha=3$ for $(k=3, r=1)$ CA rule space.
2. If a neutral network percolates then $\mathfrak{D}$, the diameter of the subgraph connecting its nodes, is larger than $\mathfrak{D}^* = \text{Diameter of configuration space}$, e.g. $\mathfrak{D}^*=64$ for RNA sequences and $\mathfrak{D}^*=27$ for $(k=3, r=1)$ CA rule space. The reverse is not necessarily correct and requires proof, herein we will provide a demonstration to this direction rather than a sound proof.


```
In[ ]:= FitnessRayPlot[{3818007614403, 3, 1}, (TestLifetime[#, 100] &)]
Out[ ]:=
```

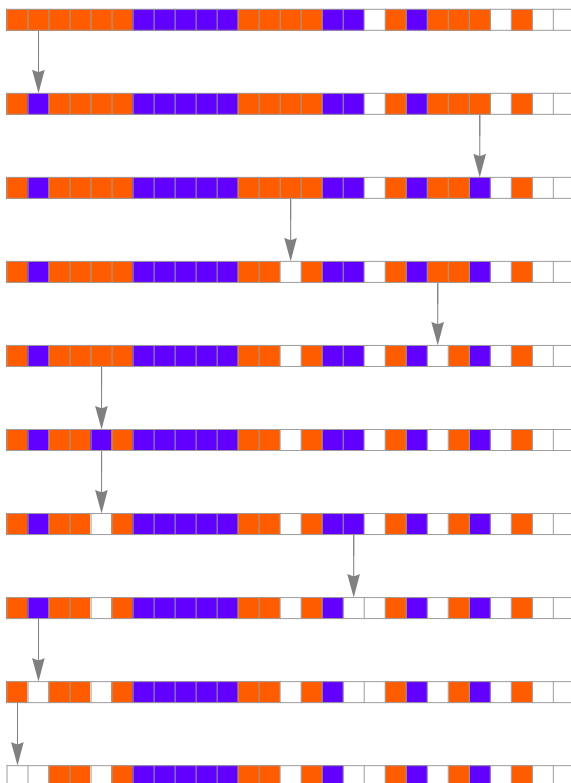


In this case the six non-neutral single-point mutation diverge to a practically infinite lifetime (denoted by dotted lines) which guarantees the continuity of the fitness-neutral set in an adaptive mutation process. In fact, such is a property of over 90% of lifetime 1 rules, which promotes the formation of large fitness-neutral sets.

Try out a to evaluate the below code a few times to get a feel of the frequency at which the initial four white bits change (the rightmost can never change):

```
In[ ]:= tenWalk = AdaptiveCellularAutomaton[<| "AdaptiveIterations" → 10, "InitialRule" →
{3818007614403, 3, 1}, "MaxSteps" → 120 |>, "RuleChanges", {"BestRule"}][[All, 1]];
RulesMutationPlot[tenWalk, "GapSize" → 3]
```

```
Out[ ]:=
```



Thus, in the scenario described above the minimal model for biological evolution features the formation of fitness-neutral sets which meet the average local neutrality criteria for percolation.

Adaptive Evolution and Average Local Neutrality

At this point, we can devise a methodology by which to test the second percolation criteria regarding network diameter and also examine what happens after the fitness-neutral sets of lifetime 1 find the “porthole” through which they escape to higher lifetimes.

The chosen methodology is random walks starting from a rule with a lifetime of 1 over which an adaptive evolution simulation is performed by the resource function `AdaptiveCellularAutomaton` for a user-defined number of iterations.

Adaptive mutation simulation - walkthrough

Pick a random rule and apply `AdaptiveCellularAutomaton`:

```
In[*]:= SeedRandom[19 670 809];
ruleWithLifetime1K3R1 = Join[Flatten[{searchForLifetimeOneRules[1]}], {3, 1}];
numIterations = 2500;
rules = AdaptiveCellularAutomaton [ <| "AdaptiveIterations" → numIterations - 1,
    "InitialRule" → ruleWithLifetime1K3R1, "MaxSteps" → 120 |>,
    "RuleChanges", {"BestRule", "BestFitness", "Index"}];
gatheredRules = GatherBy[rules, #BestFitness &];
```

Process and output run results:

```
In[*]:= lifetimeChangeData = analyseMutations[gatheredRules];
createLifetimeTable[rules, numIterations, lifetimeChangeData]
```

```
Out[*]//TableForm=
```

#	Lifetime	Iterations	Diameter	Norm_Diam	Cum_Neut_Prob	/
1	1	613	541	0.88	0.79	(
2	2	93	86	0.94	0.12	(
3	3	14	12	0.93	0.68	(
4	4	12	11	1.00	0.63	(
5	5	26	21	0.85	0.48	(
6	7	28	26	0.96	0.42	(
7	11	8	7	1.00	0.22	(
8	21	5	4	1.00	0.05	(
9	26	2	1	1.00	0.29	(
10	34	3	2	1.00	0.30	(
11	59	3	2	1.00	0.19	(
12	99	9	2	0.33	1.00	(
Active Mutations	816					
Number of Iterations	2500					
Run Efficacy	0.57					

A lot can be learned from these results:

- The large neutral network of lifetime 1 which meets both percolation criteria ($\lambda > \lambda^* = 0.58$, $\mathfrak{D} > \mathfrak{D}^* = 27$).

- Lifetime 2 also meets both percolation criteria.
- Average local neutrality decreases as lifetime increases resulting in smaller, non-percolating neutral networks.
- Run efficacy measures the amount of idle iterations - with low efficacies, such as 0.57, a high number of lifetime hops is expected throughout the simulation.
- In this example the simulation could not reach 2500 iterations because none of the lifetime 99 mutations resulted in a finite increase in lifetime. This can also be seen from the cumulative neutrality probability value of lifetime 99 which is 1, i.e. the adaptive process accepted nine lifetime conserving mutations and halted thereafter.

Adaptive Evolution and Percolation

Evidence of percolation

Thus far in this example we saw a strong indication to the existence of percolating fitness-neutral sets for two lifetimes (1 and 2).

However, as mentioned before, meeting the diameter criteria is not enough: a percolating fitness-neutral set will satisfy $\mathfrak{D} > \mathfrak{D}^* = 27$ however not every fitness-neutral set that satisfies $\mathfrak{D} > \mathfrak{D}^* = 27$ is percolating. There can exist such sets that expand in the $(k=3, r=1)$ 27D hypercube yet occupy a small portion of space therein, such sets are said to exhibit a “snake-like” structure.

In the original essay we provided the two indications below that the case of adaptive evolution simulations of the fitness-neutral sets under our lamp seems to be one of percolation.

In the **addendum** we provide a better and more convincing visualization for that.

3D visualization of the fitness-neutral set

If a fitness-neutral set is percolating we expect a dimension-reduced plot of it to be non-compact. Admittedly, this is a weaker intuition-driven indication since a 27D space is hard to grasp and also its reduction to 3D. Yet we feel it is interesting to put it on display.

Dimension-reduction for the 3D plot of the fitness-neutral set with lifetime 1 from the adaptive evolution simulation above (collapsed code):

```
In[*]:= (* 3D reduced-dimension plot for lifetime=1 only *)
```

```
ruleNumbersRw = gatheredRules[[1]][All, 1, 1];
```

```
(* Convert each decimal number to a 27-digit base 3 string *)
```

```
base3Numbers = IntegerString[#, 3, 27] & /@ ruleNumbersRw;
```

```
(* Function to convert base-3 strings to 3D coordinates *)
```

```
to3DCoordinates[str_] := Module[{digits},
```

```
  digits = ToExpression[Characters[str]];

```

```
  {Total[digits[[1 ;; 9]], Total[digits[[10 ;; 18]], Total[digits[[19 ;; 27]]}
```

```
];
```

```
(* Convert each base-3 string to 3D coordinates *)
```

```
coordinates = to3DCoordinates /@ base3Numbers;
```

```
(* Create lines based on adjacency in the list *)
```

```
lines = Line /@ Partition[coordinates, 2, 1];
```

```
(* Visualize the graph in 3D with small spheres *)
```

```
Graphics3D[{
```

```
  {Sphere[#, 0.1] & /@ coordinates},
```

```
  {Thick, lines}
```

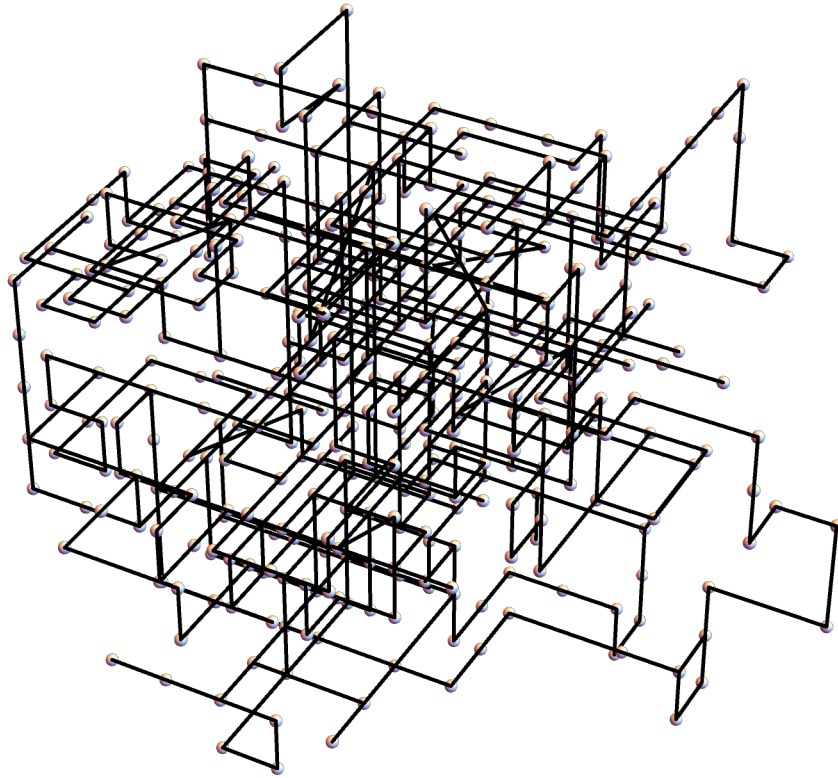
```
},
```

```
Boxed → False,
```

```
ImageSize → Large
```

```
]
```

Out[]=



Random walk, quasi-random walk and non-random walk

Apart from the random walk we demonstrated we deployed two more: a non-random walk where fitness-neutral sets are “harvested” from the multiway graph of $(k=3, r=1)$ with a breadth-first search approach, and a quasi-random walk where we still “harvest” the multiway graph yet allow random depth passages farther away in it.

‘A convex hull is the smallest convex shape that encloses a set of points in a geometric space. It can be visualized as the shape formed by stretching a rubber band around the outermost points of the set. In computational geometry, the convex hull is commonly used to simplify complex shapes and to analyze spatial properties such as the area or boundary of a dataset.’ (Wolfram Notebook Assistant)

After constructing these three fitness-neutral sets, we use `ConvexHullMesh` over a dimension reduction to 2D to estimate the “area” of the random, quasi-random and non-random fitness-neutral sets all starting from the same lifetime 1 rule.

Formation of quasi-random and non-random walks and calculation of convex hull area (collapsed code):

Note: evaluation of the second cell is time-consuming.

Code

Albeit a work-in-progress, in most adaptive evolution simulations we performed thus far data and plots demonstrate that random walks expectedly have a striking advantage in diameter and an advantage in (normalized) projected area.

Evaluate (collapsed) cell below to display data:

```
In[ ]:= TableForm[
  {
    {"Vertex/Rule Count", sizeRandomRuleRw, sizeNgQuasiRandomRule, sizeNgRandomRule},
    {"Diameter", diamRandomRuleRw, diamNgQuasiRandomRule, diamNgRandomRule},
    {"Projected Area", Round[areaDv], Round[areaQuasiNgv], Round[areaNgv]}
  },
  TableHeadings → {None, {"", "Random Walk", "Quasi-random Walk", "Non-random Walk"}}
]
```

Out[]//TableForm=

	Random Walk	Quasi-random Walk	Non-random Walk
Vertex/Rule Count	613	1304	1059
Diameter	541	14	4
Projected Area	228	240	259

Random walk, quasi-random walk and non-random walk - sample graphs

Random walk 2D graph

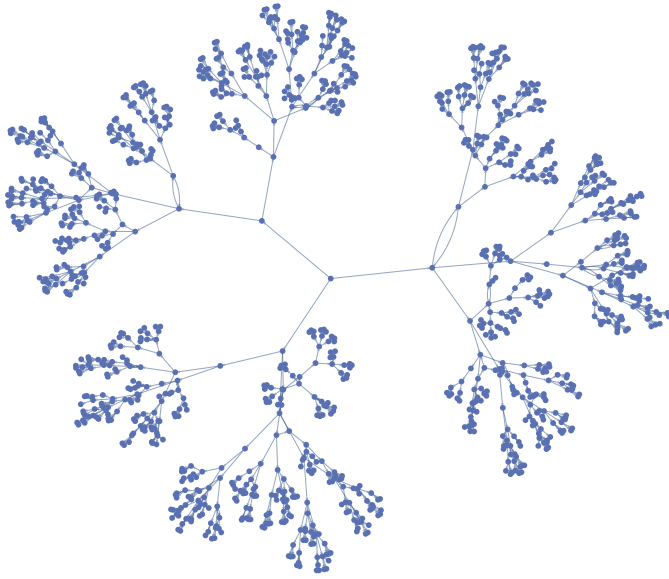
```
In[ ]:= calcDiameters[gatheredRules[[1]][[1]]]
```

Out[]=



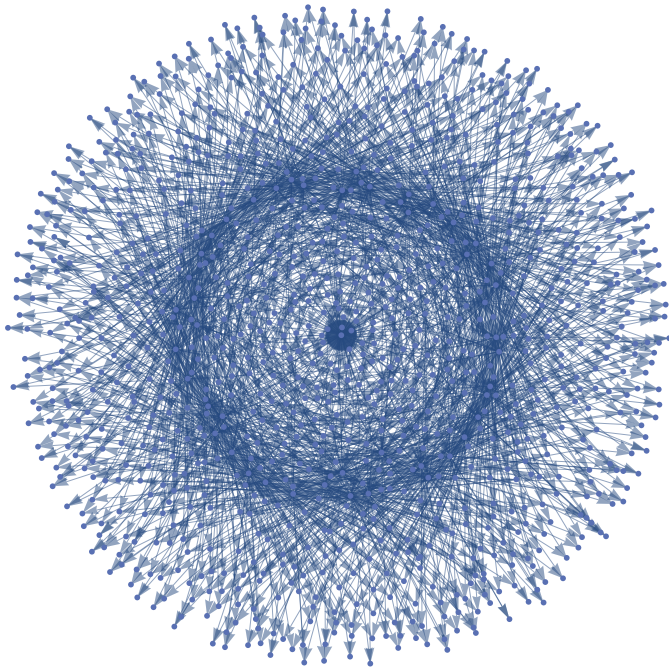
Quasi-random walk 2D graph

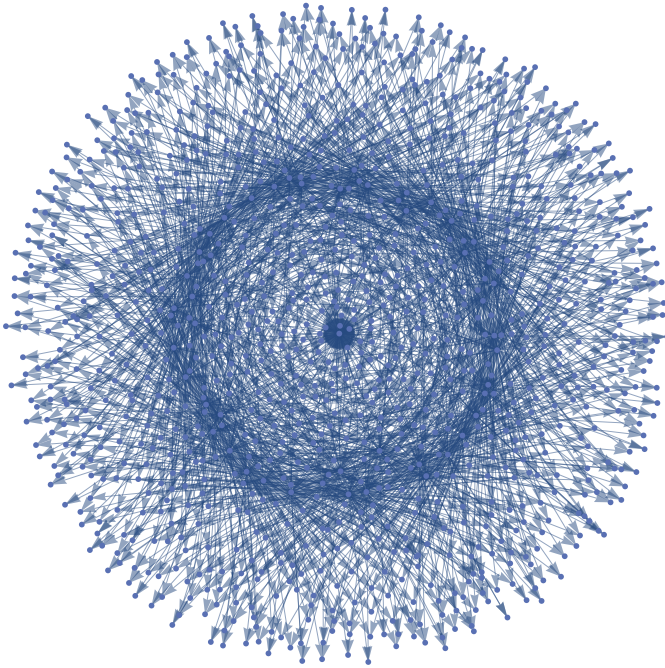
```
In[ ]:= ngQuasiRandomRule
Out[ ]:=
```



Non-random walk 2D graph

```
In[ ]:= ngRandomRule
Out[ ]:=
```





Addendum: further evidence of percolation using the random, quasi-random and non-random concept

We consider two more geometric metrics that can provide further evidence for percolation-like pattern of a set of points scattered in a hypercube:

Radius of Gyration - the root mean squared Hamming distance of points from their centroid, quantifies the spacial extent of the set.

Average Inter-point Distance - the mean Hamming distance between any pair of set members, measures the overall dispersion of the set in configuration space.

As the radius of gyration approaches its maximal Hamming diameter, 27 in our case, the set increasingly traverses the hypercube. Similarly, as the average inter-point distance (also known as sparsity) approaches its maximal value, $18/27 \approx 0.66$ in our case, the diversity of the set increases.

Taken together, these metrics can provide evidence for expansion of a fitness-neutral set in hyper-space while disfavoring narrow, snake-like propagation.

Let us examine an adaptive evolution simulation:

```
In[*]:= SeedRandom[943954];
```

```
ruleX = Join[Flatten[{searchForLifetimeOneRules[1]}], {3, 1}];
```

```
numIterationsX = 2500;
```

```
rulesX = AdaptiveCellularAutomaton[<| "AdaptiveIterations" → numIterationsX,  
  "InitialRule" → ruleX |>, "RuleChanges", {"BestRule", "BestFitness"}];
```

```

gatheredRulesX = GatherBy[rulesX, #BestFitness &];

ruleXData = gatheredRulesX[[1]];
lifetimeX = ruleXData[[1]][[2]];

sizeRandomRule = Length[ruleXData];
diamRandomRule = calcDiameters[ruleXData][[2]];

ruleXNumbers = ruleXData[[All, "BestRule", 1]];
(* Convert each decimal number to a 27-digit base 3 string *)
base3NumbersX = IntegerString[#, 3, 27] & /@ ruleXNumbers;
(* MatrixForm[base3Numbers]; *)
ptsX = Map[Characters, base3NumbersX] /. "0" → 0 /. "1" → 1 /. "2" → 2;

(* Non-random walk *)

With[{ru = ruleX}, {lt = lifetimeX},
  ngNonRandomRule = NestGraph[
    Select[getallmutations[##], TestLifetime[##, 120] == lt &] &, {ru}, 2, VertexLabels → None];

sizeNgNonRandomRule = Length[VertexList[ngNonRandomRule]];
diamNgNonRandomRule = calcDiametersNg[VertexList[ngNonRandomRule]][[1]];

ruleNRData = VertexList[ngNonRandomRule][[All, 1]];
(* Convert each decimal number to a 27-digit base 3 string *)
base3NumbersNR = IntegerString[#, 3, 27] & /@ ruleNRData;
(* MatrixForm[base3Numbers]; *)
ptsNR = Map[Characters, base3NumbersNR] /. "0" → 0 /. "1" → 1 /. "2" → 2;

(* Quasi-random walk *)

With[{ru = ruleX}, {lt = lifetimeX},
  ngQuasiRandomRule =
    NestGraph[Select[Table[RandomRuleMutation[##, 3], TestLifetime[##, 120] == lt &] &,
    {ru}, 7, VertexLabels → None];
ruleNumbersQuasiNg = VertexList[ngQuasiRandomRule][[All, 1]];
sizeNgQuasiRandomRule = Length[VertexList[ngQuasiRandomRule]];
diamNgQuasiRandomRule = calcDiametersNg[VertexList[ngQuasiRandomRule]][[1]];

ruleQRData = VertexList[ngQuasiRandomRule][[All, 1]];
(* Convert each decimal number to a 27-digit base 3 string *)
base3NumbersQR = IntegerString[#, 3, 27] & /@ ruleQRData;
(* MatrixForm[base3Numbers]; *)

```

This simulation creates a random walk fitness-neutral set of lifetime 1 of size 317 and diameter 280, a quasi-random walk of size 1370 and diameter 14 and a non-random walk of size 1059 and diame-

ter 4.

Calculating and displaying the two metrics for the above sets, it is seen that despite it is the smallest set by far the random walk traverses hyperspace the most and is the most diverse. The quasi-random walk catches up on the random walk only on radius. Expectedly, the non-random walk lags behind considerably.

```
In[ ]:= (* Radius of gyration – square root of the mean Hamming distance from centrepoint *)
RadiusOfGyration[points_] := Module[{center, n}, n = Length[points];
  center = Mean[points];
  Sqrt[1/n * Total[Map[HammingDistance[#, center] &, points]^2]]

(* Sparsity – ratio between average Hamming distance
  between any two points and maximum Hamming distance (27) *)
CalculateSparsity[points_] := Module[{distances, n, maxDist}, n = Length[points];
  maxDist = Length[points][[1]];
  distances = Mean[Flatten[DistanceMatrix[points, DistanceFunction → HammingDistance]]];
  distances/maxDist]

(* Function to calculate radius of gyration and sparsity *)
calculateMetrics[data_] := Module[{radius, sparsity},
  radius = RadiusOfGyration[data] // N;
  sparsity = CalculateSparsity[data] // N;
  {radius, sparsity}
]
```

```
VisualizePaths[proc1_, proc2_, proc3_] := Module[{path1, path2, path3, dim = 3},

  path1 = DimensionReduction[proc1, dim, Method → Automatic] /@ proc1;
  path2 = DimensionReduction[proc2, dim, Method → Automatic] /@ proc2;
  path3 = DimensionReduction[proc3, dim, Method → Automatic] /@ proc3;

  Show[

    Graphics3D[{Blue, Opacity[0.5], Line[path1], PointSize[0.007], Point[path1]}],
    Graphics3D[{Green, Opacity[0.55], Line[path2], PointSize[0.007], Point[path2]}],
    Graphics3D[{Red, Opacity[0.7], Line[path3], PointSize[0.007], Point[path3]}],

    Boxed → True, Axes → False, PlotLabel → Style["Simulations: 27D → 3D Projection", 20, Bold],
    ViewPoint → {2, -2, 1}, ImageSize → Large ]]

(* Collect metrics for each dataset *)
metricsX = calculateMetrics[ptsX];
metricsQR = calculateMetrics[ptsQR];
```

```
metricsNR = calculateMetrics[ptsNR];
```

(* Prepare the data for the table *)

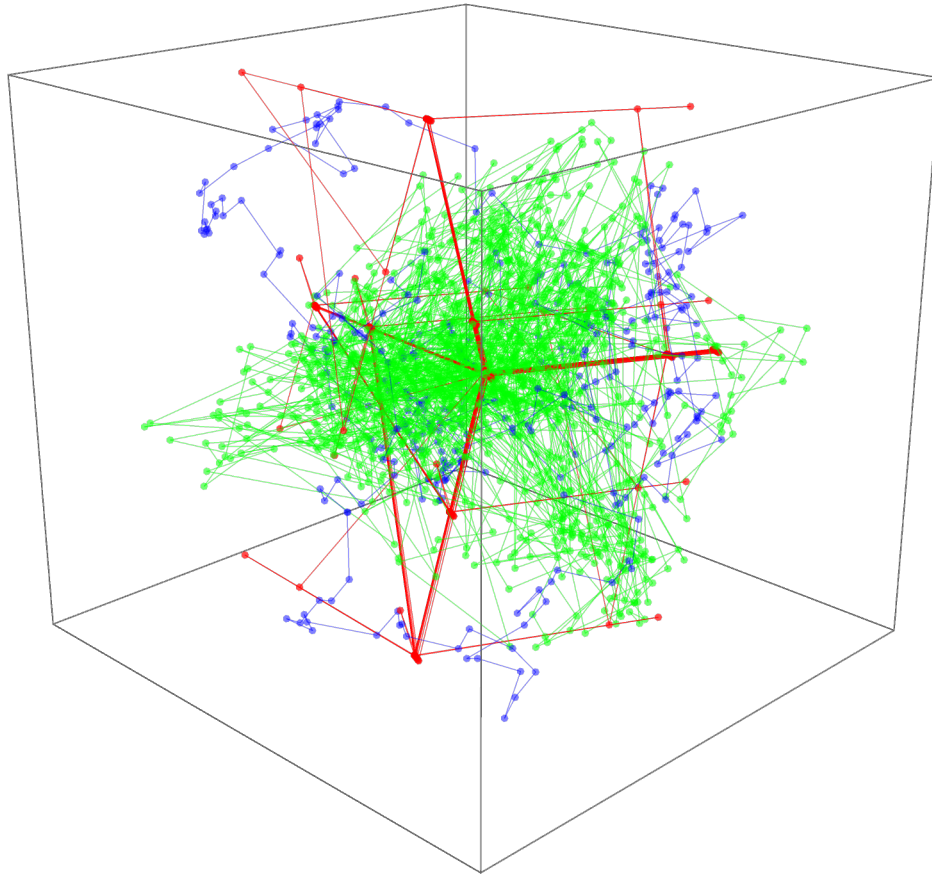
```
metricsTable = {
  {"Type", "Radius of Gyration", "Sparsity"},
  {"Random (Blue)", metricsX[[1]], metricsX[[2]]},
  {"Quasirandom (Green)", metricsQR[[1]], metricsQR[[2]]},
  {"Nonrandom (Red)", metricsNR[[1]], metricsNR[[2]]}
};
```

(* Create the visualization and table side by side *)

```
Row[{
  VisualizePaths[ptsX, ptsQR, ptsNR],
  Grid[metricsTable, Frame → All, Spacings → {2, 2}]
}]
```

Out[*n*]=

Simulations: 27D → 3D Projection



Type	Radius of Gyration	Sparsity
Random (Blue)	23.	0.524868
Quasirandom (Green)	23.	0.30318
Nonrandom (Red)	14.7802	0.135562

Concluding Remarks

This study claims that random adaptive evolution walks on a default initial condition on ($k=3$, $r=1$) seem to produce percolating fitness-neutral sets for lower lifetime values (especially 1). Much of it was done on the fly and at a high pace, yet we feel its results are a stepping stone to further experi-

ments. Implications of variable initial conditions need to be considered. More concise ways to measure distribution in 27D should be learned and implemented. Should we draw closer to the RNA case, the relevant rule space seems to be $(k=4, r=2)$ which is dozens of orders of magnitude bigger than $(k=3, r=1)$ posing computational challenges of a new level. More data from more executions of the adaptive evolution framework that is decided should be accumulated and processed.

On a personal note, i found the new way in which to do science extremely plausible. Trying to minimize the glass ceiling imposed by computational irreducibility by intuition that directed me towards making the “correct computations” (an ever-changing term) seems to be the best way to gain insight into the complex systems under research, where it seems that the rigorous tools of olden times are becoming less efficient.

Acknowledgments

I would like to thank my mentor Willem Nielsen for sharing his vast Wolfram Language knowledge with me and for challenging discussions which helped this computational study take shape. Thanks to Willem this Winter School was a highly exciting learning experience for me. I am also extending my gratitude to Stephen Wolfram for reshaping my initial thoughts on this study and directing it to new thought-provoking areas and to a first-time deployment of a new scientific thought process.

References

- [1] Wolfram (2025), “Why Does Biological Evolution Work? A Minimal Model for Biological Evolution and Other Adaptive Processes”, *Complex Systems*, 33(4), 387-488. <https://doi.org/10.25088/ComplexSystems.33.4.387>
- [2] Schuster, Fontana, Stadler and Hofacker (1994), “From Sequences to Shapes and Back: A Case Study in RNA Secondary Structures”, *Proceedings of the Royal Society of London B (Biological Sciences)*, 255, 279-284. <https://doi.org/10.1098/rspb.1994.0040>
- [3] Reidys, Stadler and Schuster (1997), “Generic properties of combinatorial maps: Neutral networks of RNA secondary structures”, *Bulletin of Mathematical Biology*, 59 (2), 339-397. <https://doi.org/10.1007/BF02462007>

CITE THIS NOTEBOOK

Percolation in fitness-neutral sets in the minimal model for biological evolution
 by Rami Zakh
 Wolfram Community, STAFF PICKS, January 15, 2026
<https://community.wolfram.com/groups/-/m/t/3607149>